实验四 综合应用设计

班级:		姓名:		学号:	
选择的设计题目:	交通灯信号器			进制/灯时:	主 14 支 90 黄 8

（支路 96+8 超过 100，因此做成 90 秒）

一、实验目的

1 掌握一般电子系统的设计、开发、调试和验证方法；
2 掌握 Verilog 语言基本语法、编程及调试方法；
3 掌握 Vivado 开发平台及 FPGA 开发板的使用。

二、实验内容

1 设计制作一个学号计数器或交通灯控制器；
2 对电路进行仿真测试，分析结果；
3 下载到 FPGA 开发板做硬件验证测试。

三、实验要求

1 画出模块的电路图；
2 分析电路的仿真波形和板卡验证结果；
3 记录设计和调试过程。

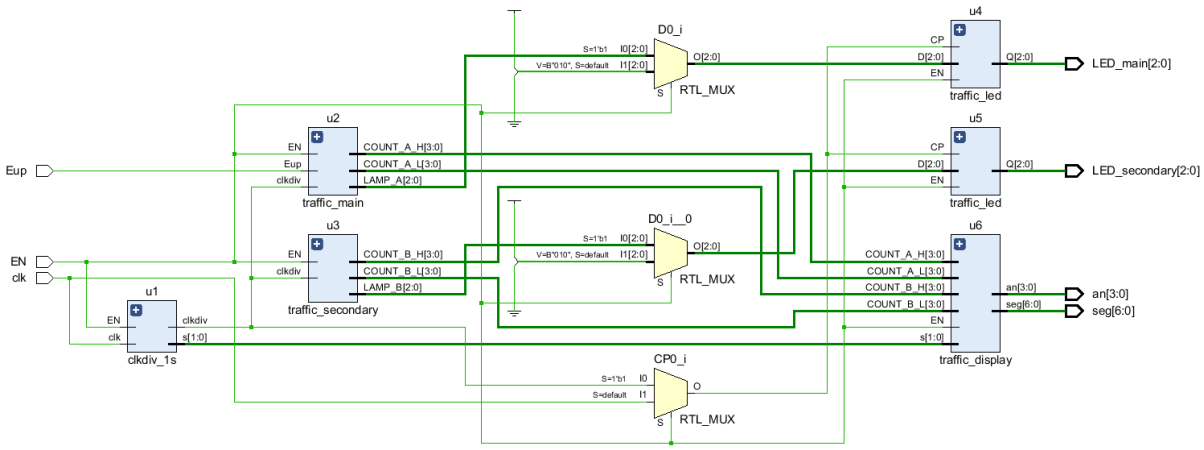
四、实验过程

对系统设计方案、模块功能、核心代码、验证结果等进行分析说明，不能只贴截图。

4.1 系统整体设计方案

1、设计目标：本实验采用 Verilog HDL 语言，利用 FPGA 实现一个交通灯控制系统，能够分别控制主路和支路的交通灯倒计时，并加入了夜间模式（黄灯闪烁）和手动控制倒计时（给主路加 3 秒）的高级功能。系统由顶层模块统一管理，调用多个子模块实现各项具体功能。系统包括时钟分频、主支路倒计时控制、LED 灯显示以及数码管动态显示模块。

2、电路图：



4.2 各模块的主要功能及实现/核心代码

1、顶层模块（traffic_top）：

（1）功能：

- 负责整体系统的协调，通过实例化子模块实现计数和灯光显示。
- 调用模块包括：时钟分频模块(clkdiv_1s)、主路控制模块(traffic_main)、支路控制模块(traffic_secondary)、LED 灯控制模块(traffic_led)和数码管显示模块(traffic_display)。
- 在调用 LED 模块时，根据使能信号：若使能有效，则进入正常倒计时模式；若无效，则进入夜间模式，让黄灯由 clk 原始时钟信号控制进行闪烁。

（2）代码：

```
//交通灯控制器顶层模块
module traffic_top(
    input       EN,                         //使能信号
    input       Eup,
    output [2:0] LED_main, LED_secondary,  //LED 灯控制信号
    output [3:0] an,                        //数码管位选信号
    input       clk,                        //时钟信号
    output [6:0] seg);                      //数码管段选信号
    wire clk_1s;        //1s 时钟信号
    wire [1:0] sel;     //数码管刷新控制信号
    //分频模块
    clkdiv_1s u1(.clk(clk), .EN(EN), .clkdiv(clk_1s), .s(sel));
    wire [3:0] COUNT_A_H, COUNT_A_L;     //主路计数高位、低位
    wire [2:0] LAMP_A;                  //主路信号灯状态
    //主路控制模块
    traffic_main u2(.clkdiv(clk_1s), .EN(EN), .Eup(Eup), .COUNT_A_H(COU
NT_A_H), .COUNT_A_L(COUNT_A_L), .LAMP_A(LAMP_A));
    wire [3:0] COUNT_B_H, COUNT_B_L;     //支路计数高位、低位
```

```
    wire [2:0] LAMP_B;                  //支路信号灯状态
    //支路控制模块
    traffic_secondary u3(.clkdiv(clk_1s), .EN(EN), .COUNT_B_H(COUNT_B_H
), .COUNT_B_L(COUNT_B_L), .LAMP_B(LAMP_B));
    //主路灯指示
    traffic_led u4(.EN(EN), .D(EN ? LAMP_A : 3'b010), .CP(EN ? clk_1s :
 clk), .Q(LED_main));
    //支路灯指示
    traffic_led u5(.EN(EN), .D(EN ? LAMP_B : 3'b010), .CP(EN ? clk_1s :
 clk), .Q(LED_secondary));
    //数码管动态显示模块
    traffic_display u6(.EN(EN), .s(sel), .COUNT_A_H(COUNT_A_H), .COUNT_
A_L(COUNT_A_L), .COUNT_B_H(COUNT_B_H), .COUNT_B_L(COUNT_B_L), .seg(seg)
, .an(an));
endmodule
```

2、时钟分频模块（clkdiv_1s）：

（1）功能：

- 将输入时钟信号分频为 1 秒时钟用于控制计时和数码管刷新。
- 使用计数器实现周期性翻转输出信号。

（2）代码：

```
//两路分频电路，一路产生 1 秒时钟信号，另一路产生数码管动态扫描信号
module clkdiv_1s(
    input clk,            //时钟信号
    input EN,             //使能信号
    output clkdiv,        //输出 1s 时钟
    output [1:0] s        //输出数码管刷新信号
);
    reg [30:0] q;         //内部计数器 q
    reg [16:0] a;         //内部计数器 a
    reg temp_clk;         //内部时钟信号
    assign s = a[16:15];          //输出数码管刷新信号
    assign clkdiv = temp_clk;     //输出 1s 时钟信号
    always @ (posedge clk)        //时钟触发
    begin
        if (EN)                    //使能有效开始计数
            if (q == 50000000-1)  //q 计数到最大
            begin
                q <= 0;                 //q 复位
                temp_clk <= ~temp_clk; //temp_clk 信号翻转
            end
            else q <= q + 1;      //计数
        else
            q <= 0;               //使能信号无效
```

```
    end
    always @ (posedge clk)        //时钟触发
        a <= a + 1;               //a 计数
endmodule
```

3、主路控制模块（traffic_main）：

（1）功能：

- 控制主路交通灯状态及倒计时功能。
- 设置倒计时逻辑与倒计时初始值，实现红、绿、黄三种灯的定时转换。
- 提供手动加时功能(Eup 信号控制)，增加主路倒计时时间，上拨一次开关给倒计时+3 秒。
- 主路灯运行模式：绿（14）-黄（8）-红（98）

（2）代码：

```
//交通灯主路控制
module traffic_main(
    input clkdiv,                    //时钟信号
    input EN,                        //使能信号
    input Eup,
    output reg[3:0] COUNT_A_H,        //主路计数高位
    output reg[3:0] COUNT_A_L,        //主路计数低位
    output reg[2:0] LAMP_A);          //主路信号灯状态
    reg[2:0] count_a;                //内部信号灯状态
    reg[3:0] num_a_h, num_a_l;        //内部计数高位，低位变量
    reg temp_a;
    reg eup_processed;               //Eup 处理标志

    always @ (posedge clkdiv) begin   //时钟触发
        COUNT_A_L <= num_a_l; COUNT_A_H <= num_a_h; //倒计时赋初值
        if (EN) begin                //使能
            if (!temp_a) begin       //temp_a 无效，开始状态转换
                eup_processed <= 0;
                temp_a <= 1;         //将 temp_a 设为有效
                case (count_a)       //根据 count_a 确定进入哪个状态
                    0: begin num_a_h <= 4'b1001; num_a_l <= 4'b1000;    //
进入红灯状态，设置倒计时初始值 98
                       LAMP_A <= 3'b100; count_a <= 2; end              //
设置信号灯状态，设置下一个状态为绿灯
                    1: begin num_a_h <= 4'b0000; num_a_l <= 4'b1000;    //
进入黄灯状态，设置倒计时初始值 8
                       LAMP_A <= 3'b010; count_a <= 0; end              //
设置信号灯状态，设置下一个状态为红灯
                    2: begin num_a_h <= 4'b0001; num_a_l <= 4'b0100;    //
进入绿灯状态，设置倒计时初始值 14
```

```
                        LAMP_A <= 3'b001; count_a <= 1; end            //
设置信号灯状态，设置下一个状态为黄灯
                    default: LAMP_A <= 3'b111;                         //
其他状态，信号灯全亮
                endcase
            end
            else begin  // temp_a 有效，开始倒计时
                if ( {num_a_h, num_a_l} > 8'b0000_0001 ) begin //倒计时>1，
继续计数
                    if (Eup && !eup_processed && {num_a_h, num_a_l} > 8
'b0000_0011) begin
                        if (num_a_l >= 4'd7) begin
                            num_a_l <= num_a_l - 4'd7;
                            num_a_h <= num_a_h + 1;
                        end
                        else begin
                            num_a_l <= num_a_l + 3;
                        end
                        eup_processed <= 1;  // 标记已处理
                    end
                    else if (num_a_l == 0) begin
                        num_a_l <= 4'b1001;             //低位置为9
                        num_a_h <= num_a_h - 1;         //高位-1
                    end
                    else begin
                        num_a_l <= num_a_l - 1;        //倒计时
                    end
                    if ( {num_a_h, num_a_l} == 8'b0000_0010 ) begin
                        temp_a <= 0;                   //提前设置temp_a，
后续进行状态转换
                    end
                end
            end
        end
        else begin
            LAMP_A <= 3'b001;
            temp_a <= 0; count_a <= 2; eup_processed <= 0;//使能信号无效，
设置信号灯初值
        end
    end
endmodule
```

4、支路控制模块（traffic_secondary)：

（1）功能：

- 控制支路交通灯状态及倒计时功能。
- 设置倒计时逻辑与倒计时初始值，实现红、绿、黄三种灯的定时转换。
- 支路灯运行模式：红（22）-绿（90）-黄（8）

（2）代码：

```
//交通灯支路控制
module traffic_secondary(
    input clkdiv,
    input EN,
    output reg[3:0] COUNT_B_H,
    output reg[3:0] COUNT_B_L,
    output reg[2:0] LAMP_B);
    reg[2:0] count_b;
    reg[3:0] num_b_h, num_b_l;
    reg temp_b;
    always @ (posedge clkdiv) begin
        COUNT_B_L <= num_b_l;
        COUNT_B_H <= num_b_h;
        if (EN) begin
            if (!temp_b) begin
                temp_b <= 1;
                case (count_b)
                    0: begin num_b_h <= 4'b0010; num_b_l <= 4'b0010;
  // 红灯：22 秒
                       LAMP_B <= 3'b100; count_b <= 2; end
                    1: begin num_b_h <= 4'b0000; num_b_l <= 4'b1000;
  // 黄灯：8 秒
                       LAMP_B <= 3'b010; count_b <= 0; end
                    2: begin num_b_h <= 4'b1001; num_b_l <= 4'b0000;
  // 绿灯：90 秒
                       LAMP_B <= 3'b001; count_b <= 1; end
                    default: LAMP_B <= 3'b111;
                endcase
            end
            else begin
                if ({num_b_h, num_b_l} > 8'b0000_0001) begin
                    if (num_b_l == 0) begin
                        num_b_l <= 4'd9;
                        num_b_h <= num_b_h - 1;
                    end else
                        num_b_l <= num_b_l - 1;
                    if ({num_b_h, num_b_l} == 8'b0000_0010) begin
                        temp_b <= 0;
```

```
                    end
                end
            end
        end
        else begin
            LAMP_B <= 3'b100;
            temp_b <= 0;
            count_b <= 0;
        end
    end
endmodule
```

5、LED 控制模块（traffic_led）：

（1）功能：

- 控制主、支路的 LED 灯状态显示。
- 在无使能状态下，实现黄灯闪烁以模拟夜间模式。

（2）代码：

```
//交通灯指示灯控制
module traffic_led(
    input EN,
    input [2:0] D,    // 灯状态输入
    input CP,         // 时钟脉冲
    output reg [2:0] Q // 灯状态输出
);
    reg [23:0] counter; // 添加计数器实现慢速闪烁

    always @(posedge CP) begin
        if (!EN && D == 3'b010) begin // 只有黄灯需要闪烁
            counter <= counter + 1;
            if (counter == 24'hFFFFFF) begin // 约0.5Hz 闪烁
                Q <= ~Q & 3'b010; // 只翻转黄灯
                counter <= 0;
            end
        end
        else begin
            Q <= D;
            counter <= 0;
        end
    end
endmodule
```

6、数码管动态显示模块（traffic_display）：

（1）功能：

- 显示主、支路当前倒计时时间。
- 利用动态扫描，实现四位数码管显示时间，夜间模式时统一显示为横杠（‘----’）。

（2）代码：

```
//交通灯动态扫描显示电路 Basys3
module traffic_display(
    input EN,
    input [1:0] s,                  //数码管刷新控制信号
    input [3:0] COUNT_A_H,          //主路倒计时高位
    input [3:0] COUNT_A_L,          //主路倒计时低位
    input [3:0] COUNT_B_H,          //支路倒计时高位
    input [3:0] COUNT_B_L,          //支路倒计时低位
    output reg[6:0] seg,            //数码管段选信号
    output reg[3:0] an              //数码管位选信号
);
    reg[3:0] digit;                 //内部变量，需显示的数值
    always @ ( * ) begin
        if (!EN) begin // 夜间模式，显示----
            an <= 4'b0000; // 所有数码管使能
            digit <= 4'b1111; // 特殊值表示"-"
        end
        else begin
            case ( s )       //根据 s 的值，分别刷新 4 位数码管
                0: begin an <= 4'b1110; digit <= COUNT_B_L; end
                1: begin an <= 4'b1101; digit <= COUNT_B_H; end
                2: begin an <= 4'b1011; digit <= COUNT_A_L; end
                3: begin an <= 4'b0111; digit <= COUNT_A_H; end
            endcase
        end
    end
    always @ ( * ) begin
        case ( digit )   //根据显示数值 digit，生成段选信号 seg
            4'b0000: seg = 7'b1000000; //显示 0
            4'b0001: seg = 7'b1111001; //显示 1
            4'b0010: seg = 7'b0100100; //显示 2
            4'b0011: seg = 7'b0110000; //显示 3
            4'b0100: seg = 7'b0011001; //显示 4
            4'b0101: seg = 7'b0010010; //显示 5
            4'b0110: seg = 7'b0000010; //显示 6
            4'b0111: seg = 7'b1111000; //显示 7
            4'b1000: seg = 7'b0000000; //显示 8
            4'b1001: seg = 7'b0010000; //显示 9
            default: seg = 7'b1111110; //默认显示-
```

```
        endcase
    end
endmodule
```

7、Basys3 约束文件：

```
## Clock signal
set_property PACKAGE_PIN W5 [get_ports clk]
    set_property IOSTANDARD LVCMOS33 [get_ports clk]

## Switches
set_property PACKAGE_PIN V17 [get_ports EN]
    set_property IOSTANDARD LVCMOS33 [get_ports EN]
set_property PACKAGE_PIN V16 [get_ports Eup]
    set_property IOSTANDARD LVCMOS33 [get_ports Eup]

## LEDs
set_property PACKAGE_PIN U16 [get_ports {LED_main[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {LED_main[0]}]
set_property PACKAGE_PIN E19 [get_ports {LED_main[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {LED_main[1]}]
set_property PACKAGE_PIN U19 [get_ports {LED_main[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {LED_main[2]}]
set_property PACKAGE_PIN V19 [get_ports {LED_secondary[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {LED_secondary[0]}]
set_property PACKAGE_PIN W18 [get_ports {LED_secondary[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {LED_secondary[1]}]
set_property PACKAGE_PIN U15 [get_ports {LED_secondary[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {LED_secondary[2]}]

##7 segment display
set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]
```

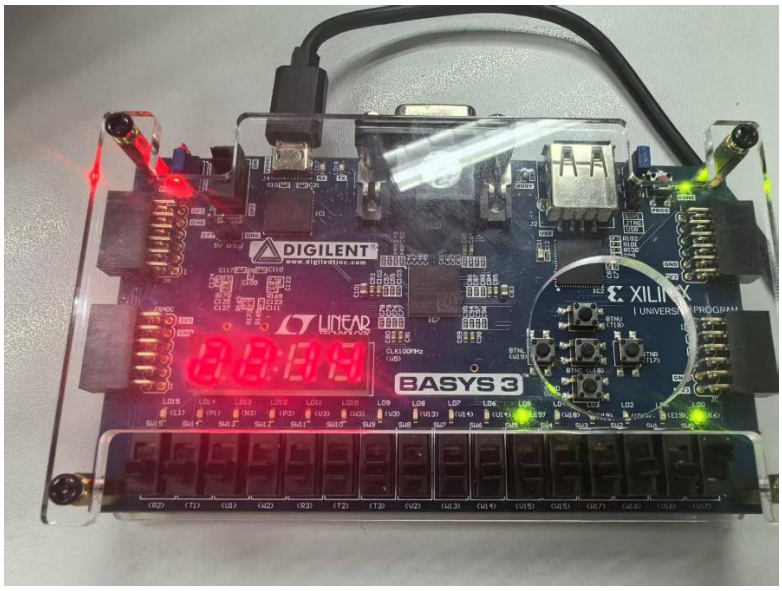
```
set_property PACKAGE_PIN U2 [get_ports {an[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN U4 [get_ports {an[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
set_property PACKAGE_PIN V4 [get_ports {an[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN W4 [get_ports {an[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
```

4.3 系统验证结果

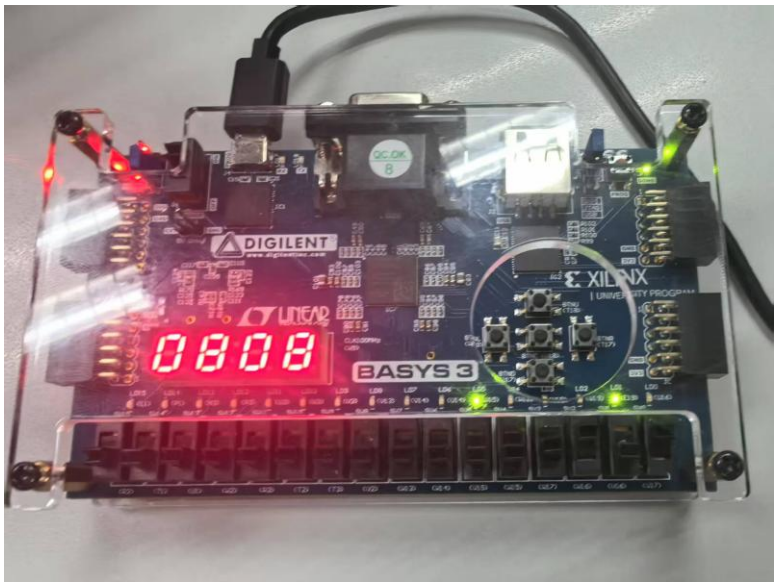
1、说明：

- 数码管左边两位为支路倒计时，右边两位为主路倒计时。
- LED 左边三个为支路灯，右边三个为主路灯，顺序都是红-黄-绿。

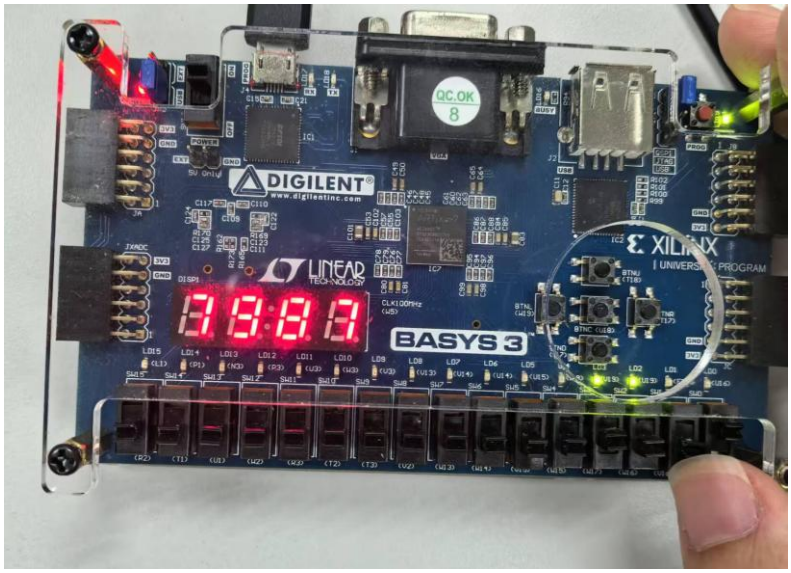
2、正常倒计时功能：



启动时，主路 14 秒绿灯，支路 22 秒红灯

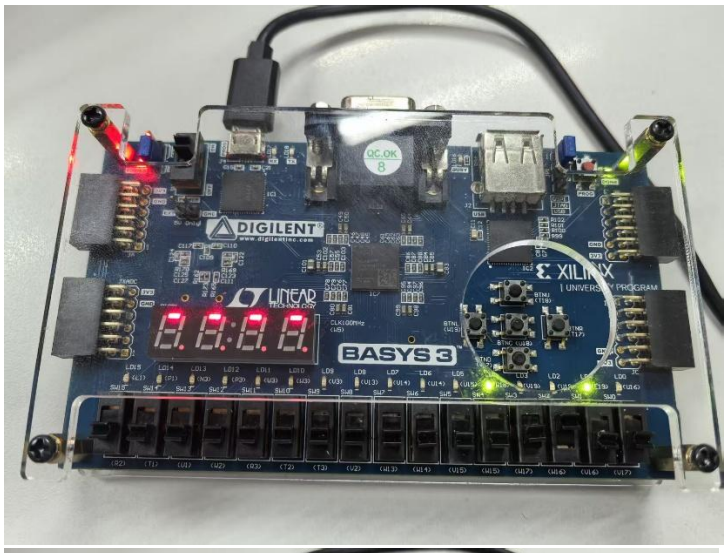


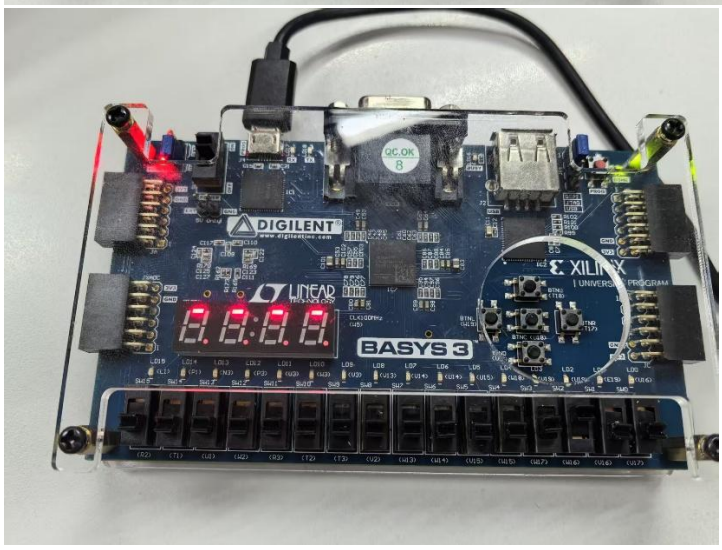
经过 14 秒后，主路黄灯亮，支路还剩 8 秒红灯



主路进入红灯（98 秒），支路进入绿灯（90 秒）

3、夜间模式：

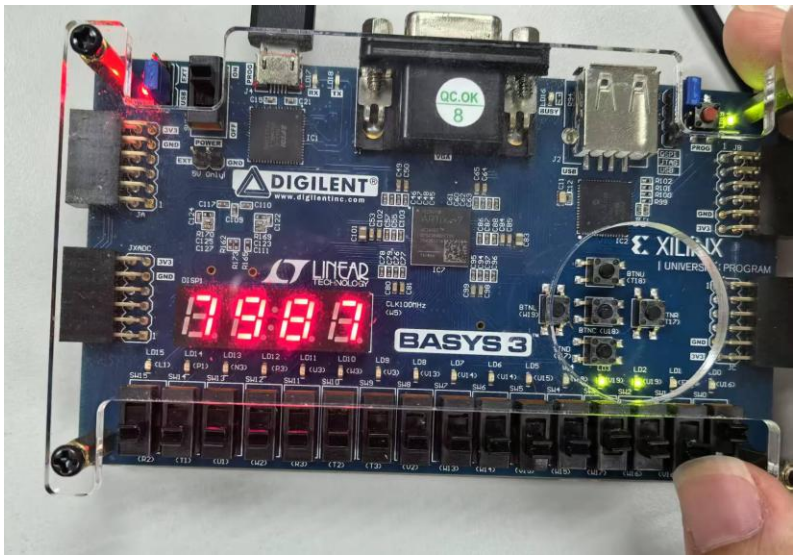


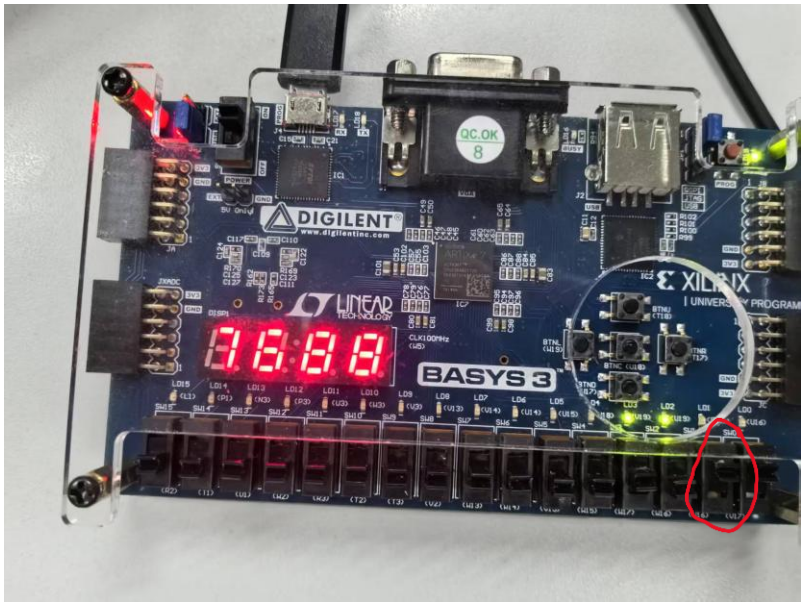


关闭使能信号 EN，主路、支路数码管都显示“-”，两个黄灯闪烁。

4、通过 Eup 信号实现手动给主路倒计时加 3 秒：

设置 V16 为 Eup 信号，当信号输入时，主路倒计时加 3 秒。





如图，左边主路倒计时在启动 Eup 后实现增加 3 秒，原本计到 76 时倒计时应为 84，因为时钟原因，增加出现一段时间的延迟，因此为 88 秒。

五、总结与收获

记录实验过程中遇到的问题和调试解决过程，心得体会等。

1、问题与调试：

（1）系统无法实现重置倒计时，在每次重启 EN 时仍从上次计时处开始计时。

调试：本意想通过 main 模块中的 else 语句部分（对应 if(EN)），但老师说这个部分只会在初始化系统时生效，遂没能完成理想中的效果。本来想将夜间模式转为黄灯放入这个部

分，最后只能在 top 中通过使能判断直接对 led 模块进行控制。

（2）在实现 Eup 对倒计时的手动控制时出现了很大的困难，启动 V16 后始终没有任何预期中的反应。

调试：通过自己写仿真文件，观察波形修改逻辑，在确认逻辑无误后认为可能是条件语句在并行过程中发生了冲突导致，遂调整了 Eup 部分的位置，最终成功实现功能。

（3）Basys3 的 display 模块与给出的 egol 版本有区别，刚开始时倒计时无法正常显示。

调试：观察倒计时异常，发现 basys3 的 7-seg 与 an 的设置其实与 egol 完全相反，是低有效的系统，因此将 0、1 翻转并调整了 an 的位数后顺利解决问题。

2、心得体会：

本实验过程中，我深刻掌握了 FPGA 的开发设计流程，体会到模块化设计思想的重要性。过程中被手动加时的逻辑问题困扰许久，通过反复仿真调试并仔细分析状态转换逻辑最终成功解决，极大地增强了我对 Verilog 语法的熟练度与理解。此次实验增强了对 Verilog HDL 编程的理解和 FPGA 平台的熟练使用能力。